Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №10

Дисциплина: Основы программирования на Python Тема: HTTP и REST API в Python: разработка клиент-серверного веб-приложения

Вариант 2: Библиотека

Цель работы

Изучить принципы работы протокола HTTP и получить практические навыки разработки и использования REST API на Python.

В ходе работы необходимо изучить устройство HTTP-запроса и HTTP-ответа, разобраться с методами GET, POST, PUT, PATCH и DELETE, изучить HTTP-заголовки, параметры и тело запроса, познакомиться с кодами состояния HTTP, изучить принципы REST, разработать собственный HTTP API, реализовать клиент для взаимодействия с API, выполнять запросы из Python с использованием библиотеки requests, реализовать сериализацию данных в JSON, организовать обработку ошибок, документировать разработанный API и протестировать его.

Постановка индивидуального задания (Вариант 2. Библиотека)

Предметная область варианта — библиотека, основной ресурс — книга (Book). Требуется разработать REST API библиотечного фонда и отдельную программу-клиент, работающую с этим API по протоколу HTTP.

Модель Book содержит семь полей: идентификатор id, название title, автор author, жанр genre, год издания year, цена экземпляра price и количество доступных экземпляров available. Поля снабжены аннотациями типов, ресурс реализован через декоратор dataclass.

Обязательные операции над ресурсом:

– GET /books — получить список книг;

– GET /books/{id} — получить одну книгу по идентификатору;

– POST /books — создать книгу;

– PUT /books/{id} — полностью изменить книгу;

– PATCH /books/{id} — частично изменить книгу;

– DELETE /books/{id} — удалить книгу.

Специализированная операция варианта — поиск книг по автору: GET /books/search?author=... . Дополнительно реализованы фильтрация по жанру, автору, цене, году издания и наличию экземпляров, а также сортировка по нескольким полям.

Данные хранятся в памяти процесса: использование базы данных, ORM, внешних API и готовых CRUD-шаблонов в основной части работы запрещено. Бизнес-логика не размещается в HTTP-обработчиках: запрос проходит через слои Route → Service → Repository.

Краткие теоретические сведения

HTTP (HyperText Transfer Protocol) — протокол прикладного уровня, предназначенный для обмена данными между клиентом и сервером. Клиент формирует запрос, сервер обрабатывает его и возвращает ответ. Протокол работает поверх TCP: сначала устанавливается TCP-соединение, затем в нём передаются текстовые HTTP-сообщения.

HTTP-запрос состоит из строки запроса (метод, путь ресурса и версия протокола), заголовков, пустой строки и, при необходимости, тела. Например, запрос на создание книги содержит метод POST, путь /books, заголовок Content-Type: application/json и JSON-тело с полями книги. HTTP-ответ устроен аналогично: строка состояния с кодом, заголовки и тело ответа.

Метод HTTP определяет действие над ресурсом. GET предназначен для получения данных и не должен изменять состояние сервера; POST создаёт новый ресурс; PUT полностью заменяет представление ресурса; PATCH изменяет только переданные поля; DELETE удаляет ресурс. Методы GET, PUT и DELETE

идемпотентны: повторный запрос с теми же данными приводит к тому же состоянию сервера.

Код состояния HTTP сообщает результат обработки запроса. В работе используются коды, приведённые в таблице 1.

Код	Название	Когда возвращается
200	OK	успешное получение или изменение ресурса
201	Created	ресурс создан, в заголовке Location — его адрес
204	No Content	успешное удаление, тело ответа отсутствует
400	Bad Request	некорректные данные, идентификатор или параметры
404	Not Found	ресурс с указанным идентификатором отсутствует
405	Method Not Allowed	метод не поддерживается для данного ресурса
500	Internal Server Error	внутренняя ошибка сервера

Таблица 1 — Коды состояния HTTP, используемые в разработанном API

Данные между клиентом и сервером передаются в формате JSON. В Python словарь преобразуется в JSON функцией json.dumps, обратное преобразование выполняет json.loads; библиотека requests позволяет передавать словарь напрямую через параметр json. Заголовок Content-Type: application/json сообщает серверу формат тела запроса, заголовок Accept — желаемый формат ответа.

REST — архитектурный стиль построения распределённых систем. Система рассматривается как набор ресурсов, каждый ресурс имеет уникальный URI, действия над ресурсами выполняются стандартными HTTP-методами, а взаимодействие является stateless: каждый запрос содержит всю информацию, необходимую для его обработки, и сервер не хранит состояние клиентской сессии между независимыми запросами.

Адрес ресурса строится по принципу «URI описывает ресурс, метод — действие». Коллекция книг находится по адресу /books, конкретная книга — по адресу /books/{id}. Идентификатор ресурса передаётся path-параметром, а фильтры, сортировка и поиск — query-параметрами, например /books?genre=роман&sort=price&order=desc.

Архитектура приложения

Приложение разделено на слои. HTTP-клиент обращается к Flask-приложению, контроллеры разбирают параметры запроса и вызывают сервис, сервис реализует бизнес-логику и работает с репозиторием, репозиторий хранит объекты в памяти.

```
HTTP Client (client.py)
      |  HTTP, JSON
      v
Flask API (app.py, routes.py)
      |  контроллеры: разбор запроса, коды ответа
      v
Service (service.py)
      |  валидация, фильтрация, сортировка, поиск
      v
Repository (repository.py)
      |  хранение объектов в памяти
      v
Data (dict[int, Book])
```

Проект имеет следующую структуру:

```
lab_10/
├── app.py            сборка приложения, обработчики ошибок, запуск
├── routes.py         контроллеры REST API (Blueprint)
├── models.py         модель Book (dataclass)
├── repository.py     хранилище книг в памяти
├── service.py        бизнес-логика библиотечного фонда
├── schemas.py        валидация данных и разбор параметров
├── exceptions.py     исключения и единый формат ошибок
├── client.py         консольный HTTP-клиент
└── tests/
    ├── test_api.py          тесты HTTP-уровня
    ├── test_service.py      тесты бизнес-логики
    └── test_repository.py   тесты хранилища
```

Контроллер не содержит бизнес-логики: он проверяет path-параметр, вызывает соответствующий метод сервиса и формирует HTTP-ответ. Валидация входных данных вынесена в модуль schemas.py, исключения — в exceptions.py, что позволяет возвращать ошибки в едином формате для всех endpoint.

Описание API

Разработанный API содержит шесть обязательных операций над ресурсом и одну специализированную операцию поиска. Полный перечень приведён в таблице 2.

Метод	Endpoint	Назначение	Успешный код
GET	/books	список книг, фильтрация, сортировка	200

Метод	Endpoint	Назначение	Успешный код
GET	/books/{id}	книга по идентификатору	200
POST	/books	создание книги	201
PUT	/books/{id}	полное изменение книги	200
PATCH	/books/{id}	частичное изменение книги	200
DELETE	/books/{id}	удаление книги	204
GET	/books/search?author=	поиск книг по автору	200

Таблица 2 — Endpoint разработанного REST API

Параметры фильтрации и сортировки списка: author — подстрока в имени автора, genre — точное совпадение жанра, available — наличие экземпляров (true или false), min_price и max_price — диапазон цены, year_from и year_to — диапазон года издания, sort — поле сортировки (title, author, genre, year, price, available), order — направление сортировки (asc или desc).

Все ошибки возвращаются в едином формате: поле error содержит текстовое описание, поле code — машиночитаемый код ошибки, например BOOK_NOT_FOUND, VALIDATION_ERROR, INVALID_REQUEST, NOT_FOUND, METHOD_NOT_ALLOWED.

Примеры HTTP-запросов

Ниже приведены примеры запросов ко всем основным endpoint и соответствующие ответы сервера.

Получение списка книг:

```
GET /books HTTP/1.1
Host: 127.0.0.1:5000
Accept: application/json


HTTP/1.1 200 OK
Content-Type: application/json


[
  {"id": 1, "title": "Мастер и Маргарита",
   "author": "Булгаков М.А.", "genre": "роман",
   "year": 1967, "price": 750.0, "available": 3},
  {"id": 2, "title": "Война и мир",
   "author": "Толстой Л.Н.", "genre": "роман",
   "year": 1869, "price": 1200.0, "available": 2}
]
```

Получение книги по идентификатору:

```
GET /books/1 HTTP/1.1
```

```
Host: 127.0.0.1:5000


HTTP/1.1 200 OK


{"id": 1, "title": "Мастер и Маргарита",
 "author": "Булгаков М.А.", "genre": "роман",
 "year": 1967, "price": 750.0, "available": 3}
```

Создание книги:

```
POST /books HTTP/1.1
Host: 127.0.0.1:5000
Content-Type: application/json


{"title": "Гарри Поттер и философский камень",
 "author": "Роулинг Дж.К.", "genre": "фэнтези",
 "year": 1997, "price": 890, "available": 5}


HTTP/1.1 201 Created
Location: /books/6


{"id": 6, "title": "Гарри Поттер и философский камень",
 "author": "Роулинг Дж.К.", "genre": "фэнтези",
 "year": 1997, "price": 890.0, "available": 5}
```

Полное изменение книги (PUT) и частичное изменение (PATCH):

```
PUT /books/6 HTTP/1.1
Content-Type: application/json


{"title": "Гарри Поттер и Тайная комната",
 "author": "Роулинг Дж.К.", "genre": "фэнтези",
 "year": 1998, "price": 940, "available": 4}


HTTP/1.1 200 OK


PATCH /books/6 HTTP/1.1
Content-Type: application/json


{"price": 899}


HTTP/1.1 200 OK   (изменена только цена, остальные поля сохранены)
```

Удаление книги:

```
DELETE /books/6 HTTP/1.1
Host: 127.0.0.1:5000


HTTP/1.1 204 No Content
(тело ответа отсутствует)
```

Специализированная операция — поиск по автору:

```
GET /books/search?author=булгаков HTTP/1.1
Host: 127.0.0.1:5000


HTTP/1.1 200 OK


{"author": "булгаков", "count": 2, "items": [
  {"id": 1, "title": "Мастер и Маргарита", ...},
```

```
  {"id": 4, "title": "Собачье сердце", ...}
]}
```

Обработка ошибок:

```
GET /books/999 HTTP/1.1          ->  HTTP/1.1 404 Not Found

{"error": "Book with id=999 not found",
 "code": "BOOK_NOT_FOUND"}

POST /books  (name пустое, price = -100)
                                 ->  HTTP/1.1 400 Bad Request

{"error": "Field 'title' must not be empty",
 "code": "VALIDATION_ERROR"}

DELETE /books                    ->  HTTP/1.1 405 Method Not Allowed
```

Реализация

Модель предметной области описана классом Book с аннотациями типов. Метод to_dict преобразует объект в словарь для сериализации в JSON.

```
from dataclasses import asdict, dataclass



@dataclass
class Book:
    """Книга библиотечного фонда."""


    id: int
    title: str
    author: str
    genre: str
    year: int
    price: float
    available: int


    def to_dict(self) -> dict:
        """Представление книги в виде словаря."""
        return asdict(self)
```

Хранилище реализовано классом BookRepository и хранит книги в словаре, где ключом является идентификатор. Идентификаторы назначаются автоматически при создании записи.

```
class BookRepository:
    """Хранилище книг в памяти."""


    def __init__(self, books: list[Book] | None = None) -> None:
        self._books: dict[int, Book] = {}
        self._next_id: int = 1
        for book in books or []:
            self.create(book)


    def get_all(self) -> list[Book]:
```

```
        return list(self._books.values())

    def get_by_id(self, book_id: int) -> Book | None:
        return self._books.get(book_id)

    def create(self, book: Book) -> Book:
        book.id = self._next_id
        self._books[book.id] = book
        self._next_id += 1
        return book

    def update(self, book: Book) -> Book:
        if book.id not in self._books:
            raise KeyError(book.id)
        self._books[book.id] = book
        return book

    def delete(self, book_id: int) -> bool:
        return self._books.pop(book_id, None) is not None
```

Сервис содержит бизнес-логику: проверку существования книги, валидацию данных, фильтрацию, сортировку и поиск по автору. При отсутствии книги возбуждается исключение NotFoundError, которое на уровне приложения преобразуется в ответ 404.

```
class BookService:

    def __init__(self, repository: BookRepository) -> None:
        self.repository = repository

    def list_books(self, filters: dict, sort_by: str | None,
                   order: str) -> list[Book]:
        books = [book for book in self.repository.get_all()
                 if self._matches(book, filters)]
        if sort_by:
            books.sort(key=lambda book: getattr(book, sort_by),
                       reverse=order == "desc")
        return books

    def get_book(self, book_id: int) -> Book:
        book = self.repository.get_by_id(book_id)
        if book is None:
            raise NotFoundError(
                f"Book with id={book_id} not found")
        return book

    def create_book(self, payload: dict) -> Book:
        data = validate_book(payload)
        return self.repository.create(Book(id=0, **data))

    def patch_book(self, book_id: int, payload: dict) -> Book:
        book = self.get_book(book_id)
        changes = validate_book(payload, partial=True)
        updated = Book(
            id=book.id,
            title=changes.get("title", book.title),
            author=changes.get("author", book.author),
            genre=changes.get("genre", book.genre),
            year=changes.get("year", book.year),
            price=changes.get("price", book.price),
            available=changes.get("available", book.available),
```

```
        )
        return self.repository.update(updated)

    def search_by_author(self, author: str) -> list[Book]:
        query = author.strip().lower()
        books = [book for book in self.repository.get_all()
                 if query in book.author.lower()]
        books.sort(key=lambda book: (book.author.lower(),
                                     book.title.lower()))
        return books
```

Контроллеры реализованы через Blueprint. Обработчик принимает path-параметр, проверяет его, вызывает метод сервиса и возвращает JSON-ответ с нужным кодом состояния.

```
books_api = Blueprint("books", __name__, url_prefix="/books")



@books_api.get("")
def list_books() -> Response:
    filters = parse_filters(request.args)
    sort_by, order = parse_sorting(request.args)
    books = get_service().list_books(filters, sort_by, order)
    return jsonify([book.to_dict() for book in books])



@books_api.get("/search")
def search_books() -> Response:
    author = request.args.get("author", "")
    books = get_service().search_by_author(author)
    return jsonify({"author": author, "count": len(books),
                    "items": [b.to_dict() for b in books]})



@books_api.post("")
def create_book() -> tuple[Response, int]:
    book = get_service().create_book(request.get_json(silent=True))
    response = jsonify(book.to_dict())
    response.headers["Location"] = f"/books/{book.id}"
    return response, 201



@books_api.patch("/<book_id>")
def patch_book(book_id: str) -> Response:
    book = get_service().patch_book(parse_book_id(book_id),
                                    request.get_json(silent=True))
    return jsonify(book.to_dict())



@books_api.delete("/<book_id>")
def delete_book(book_id: str) -> Response:
    get_service().delete_book(parse_book_id(book_id))
    return Response(status=204)
```

Ошибки обрабатываются централизованно в модуле app.py: все исключения приложения наследуются от ApiError и преобразуются в JSON

единого формата, для неизвестного адреса и неподдерживаемого метода также заданы обработчики.

```
class ApiError(Exception):
    status_code: int = 500
    code: str = "INTERNAL_ERROR"
    message: str = "Internal server error"


    def to_dict(self) -> dict:
        return {"error": self.message, "code": self.code}



class NotFoundError(ApiError):
    status_code = 404
    code = "BOOK_NOT_FOUND"



class ValidationError(ApiError):
    status_code = 400
    code = "VALIDATION_ERROR"
```

```
def register_error_handlers(app: Flask) -> None:
    @app.errorhandler(ApiError)
    def handle_api_error(error: ApiError):
        return jsonify(error.to_dict()), error.status_code


    @app.errorhandler(404)
    def handle_not_found(error):
        return jsonify({"error": "Resource not found",
                        "code": "NOT_FOUND"}), 404


    @app.errorhandler(405)
    def handle_method_not_allowed(error):
        return jsonify({"error": "Method not allowed",
                        "code": "METHOD_NOT_ALLOWED"}), 405
```

Клиент — отдельная программа client.py, использующая библиотеку requests. Класс ApiClient инкапсулирует адрес сервера и формирование запросов, а консольное меню реализует все операции, предусмотренные заданием.

```
class ApiClient:


    def __init__(self, base_url: str = DEFAULT_URL,
                 timeout: float = 5.0) -> None:
        self.base_url = base_url.rstrip("/")
        self.timeout = timeout


    def _request(self, method: str, path: str,
                 **kwargs) -> tuple[int, object]:
        url = f"{self.base_url}{path}"
        try:
            response = requests.request(method, url,
                                        timeout=self.timeout, **kwargs)
        except requests.ConnectionError:
            return 0, {"error": "Сервер недоступен",
                       "code": "CONNECTION_ERROR"}
        if response.status_code == 204:
            return 204, None
        return response.status_code, response.json()
```

Тестирование

Автоматическое тестирование выполнено с использованием pytest. Написано 19 тестов: два теста репозитория, пять тестов сервиса и двенадцать тестов HTTP-уровня. Тесты API выполняются на встроенном тестовом клиенте Flask, поэтому запускать сервер для них не требуется.

Тесты репозитория проверяют автоматическое назначение идентификаторов и удаление записи. Тесты сервиса проверяют создание книги с валидацией, отказ при некорректных данных, исключение при обращении к отсутствующей книге, частичное изменение и совместную работу фильтрации и сортировки.

Тесты API проверяют получение списка и отдельной книги, создание с кодом 201 и заголовком Location, полное и частичное изменение, удаление с кодом 204, ошибку 404, ошибку 400 при некорректных данных и при нечисловом идентификаторе, фильтрацию, сортировку, специализированную операцию поиска, а также коды 404 и 405 для неизвестного адреса и неподдерживаемого метода.

Результат выполнения тестов приведён на рисунке 10.1: все 19 тестов пройдены успешно.

```
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\gorak\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
rootdir: C:\elkin\lab_10
collecting ... collected 19 items

tests/test_api.py::test_get_books_returns_list PASSED                    [  5%]
tests/test_api.py::test_get_book_by_id PASSED                            [ 10%]
tests/test_api.py::test_get_missing_book_returns_404 PASSED              [ 15%]
tests/test_api.py::test_create_book_returns_201 PASSED                   [ 21%]
tests/test_api.py::test_create_book_with_invalid_data_returns_400 PASSED [ 26%]
tests/test_api.py::test_put_replaces_book PASSED                         [ 31%]
tests/test_api.py::test_patch_updates_price PASSED                       [ 36%]
tests/test_api.py::test_delete_book_returns_204 PASSED                   [ 42%]
tests/test_api.py::test_invalid_identifier_returns_400 PASSED            [ 47%]
tests/test_api.py::test_filter_and_sort_books PASSED                     [ 52%]
tests/test_api.py::test_search_by_author_endpoint PASSED                 [ 57%]
tests/test_api.py::test_unknown_route_and_method PASSED                  [ 63%]
tests/test_repository.py::test_create_assigns_incremental_ids PASSED     [ 68%]
tests/test_repository.py::test_delete_removes_book_and_reports_missing PASSED [ 73%]
tests/test_service.py::test_create_book_validates_and_stores PASSED      [ 78%]
tests/test_service.py::test_create_book_rejects_invalid_data PASSED      [ 84%]
tests/test_service.py::test_get_missing_book_raises_not_found PASSED     [ 89%]
tests/test_service.py::test_patch_changes_only_given_fields PASSED       [ 94%]
tests/test_service.py::test_list_books_filters_and_sorts PASSED          [100%]

============================== 19 passed in 0.48s ==============================
```

Рисунок 10.1 — Результат выполнения автоматических тестов pytest

Демонстрация работы приложения

Сервер запускается командой python app.py и принимает подключения по адресу 127.0.0.1:5000. Журнал сервера (рисунок 10.2) показывает запуск приложения и строки запросов с методами, адресами и кодами ответов.

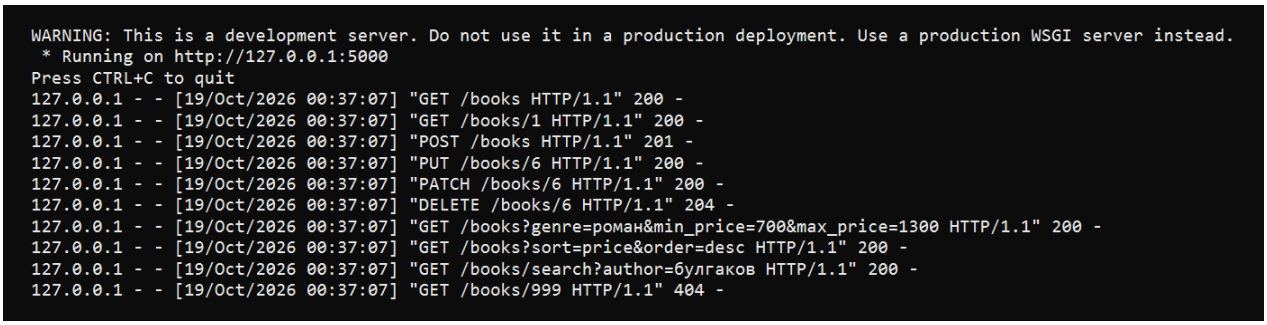


Рисунок 10.2 — Запуск сервера и журнал обработанных HTTP-запросов

Клиент запускается командой python client.py и выводит консольное меню с девятью операциями и выходом (рисунок 10.3).

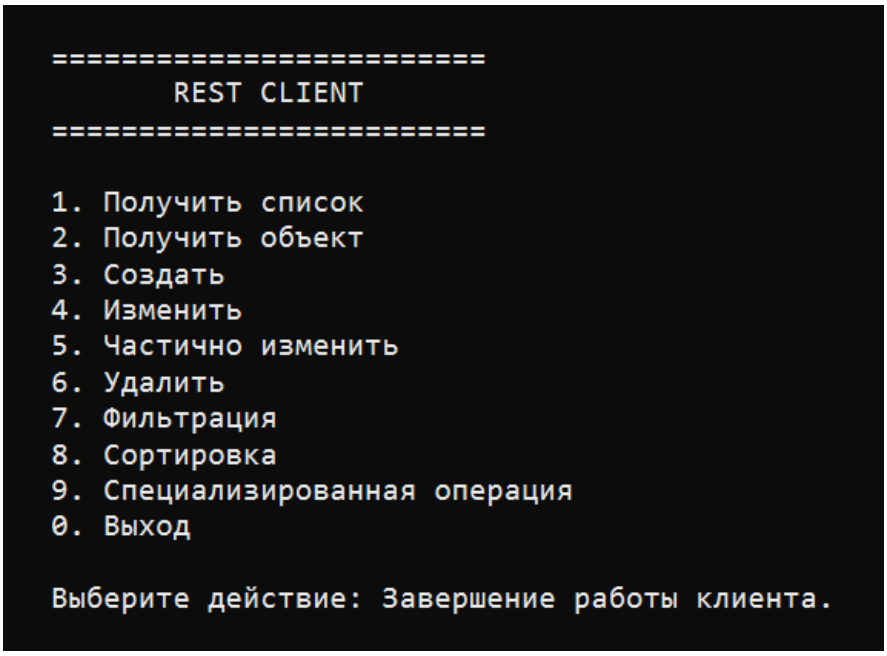


Рисунок 10.3 — Главное меню консольного HTTP-клиента

На рисунке 10.4 показано получение списка книг и получение одной книги по идентификатору. Клиент выводит код ответа и содержимое JSON-ответа.

```
=== 1. Список книг ===
HTTP 200
  #1 Мастер и Маргарита — Булгаков М.А., роман, 1967, 750.0 руб., доступно: 3
  #2 Война и мир — Толстой Л.Н., роман, 1869, 1200.0 руб., доступно: 2
  #3 Преступление и наказание — Достоевский Ф.М., роман, 1866, 640.0 руб., доступно: 0
  #4 Собачье сердце — Булгаков М.А., повесть, 1925, 410.0 руб., доступно: 4
  #5 Пикник на обочине — Стругацкие А. и Б., фантастика, 1972, 520.0 руб., доступно: 1

=== 2. Книга с id=1 ===
HTTP 200
  id: 1
  title: Мастер и Маргарита
  author: Булгаков М.А.
  genre: роман
  year: 1967
  price: 750.0
  available: 3
```

Рисунок 10.4 — Получение списка книг и книги с идентификатором 1

Создание книги (POST) и полное изменение (PUT) приведены на рисунке 10.5. При создании сервер возвращает код 201, при полном изменении — код 200.

```
=== 3. Создание книги ===
HTTP 201
  id: 6
  title: Гарри Поттер и философский камень
  author: Роулинг Дж.К.
  genre: фэнтези
  year: 1997
  price: 890.0
  available: 5

=== 4. Полное изменение книги с id=6 (PUT) ===
HTTP 200
  id: 6
  title: Гарри Поттер и Тайная комната
  author: Роулинг Дж.К.
  genre: фэнтези
  year: 1998
  price: 940.0
  available: 4
```

Рисунок 10.5 — Создание книги и полное изменение книги методом PUT

Частичное изменение (PATCH) и удаление (DELETE) показаны на рисунке 10.6: изменяются только цена и количество экземпляров, удаление возвращает код 204 без тела ответа.

```
=== 5. Частичное изменение книги с id=6 (PATCH) ===
HTTP 200
  id: 6
  title: Гарри Поттер и Тайная комната
  author: Роулинг Дж.К.
  genre: фэнтези
  year: 1998
  price: 899.0
  available: 7

=== 6. Удаление книги с id=6 (DELETE) ===
HTTP 204
(тело ответа отсутствует)
```

Рисунок 10.6 — Частичное изменение книги и удаление книги

Фильтрация и сортировка выполняются query-параметрами: на рисунке 10.7 выбраны книги жанра «роман» с ценой от 700 до 1300 рублей и выполнена сортировка по цене по убыванию.

```
=== 7. Фильтрация: жанр 'роман', цена от 700 до 1300 ===
HTTP 200
  #1 Мастер и Маргарита — Булгаков М.А., роман, 1967, 750.0 руб., доступно: 3
  #2 Война и мир — Толстой Л.Н., роман, 1869, 1200.0 руб., доступно: 2

=== 8. Сортировка: price, desc ===
HTTP 200
  #2 Война и мир — Толстой Л.Н., роман, 1869, 1200.0 руб., доступно: 2
  #1 Мастер и Маргарита — Булгаков М.А., роман, 1967, 750.0 руб., доступно: 3
  #3 Преступление и наказание — Достоевский Ф.М., роман, 1866, 640.0 руб., доступно: 0
  #5 Пикник на обочине — Стругацкие А. и Б., фантастика, 1972, 520.0 руб., доступно: 1
  #4 Собачье сердце — Булгаков М.А., повесть, 1925, 410.0 руб., доступно: 4
```

Рисунок 10.7 — Фильтрация по жанру и цене, сортировка по цене

Специализированная операция варианта — поиск книг по автору. На рисунке 10.8 поиск по строке «булгаков» возвращает две книги; там же показан ответ сервера с ошибкой 404 при обращении к отсутствующей книге.

```
=== 9. Поиск по автору: 'булгаков' ===
HTTP 200
  author: булгаков
  count: 2
  items:
    #1 Мастер и Маргарита — Булгаков М.А., роман, 1967, 750.0 руб., доступно: 3
    #4 Собачье сердце — Булгаков М.А., повесть, 1925, 410.0 руб., доступно: 4

=== Ошибка 404: книга с id=999 ===
HTTP 404
  error: Book with id=999 not found
  code: BOOK_NOT_FOUND
```

Рисунок 10.8 — Поиск книг по автору и обработка ошибки 404

Веб-страница каталога

REST API возвращает данные в формате JSON, и HTTP-клиентом может быть не только программа на Python, но и браузер. Чтобы продемонстрировать это, поверх разработанного API добавлена витрина каталога: маршрут GET / отдаёт шаблон templates/index.html, который загружает данные через те же endpoints GET /books и GET /books/{id}, что и консольная программа client.py. Таким образом, страница не содержит собственной логики работы с данными — вся бизнес-логика по-прежнему находится в слое Service, а хранилище — в Repository.

На странице реализованы поиск по названию или автору, фильтр «только доступные» (query-параметр available=true) и сортировка по названию, автору, цене или году (sort и order). При выборе карточки браузер выполняет запрос GET /books/{id} и показывает детали книги. Это подтверждает важный вывод лабораторной работы: любой HTTP-клиент — консольная программа, тест или браузер — работает с разработанным API одинаково, с одним набором endpoints, методов и кодов состояния.

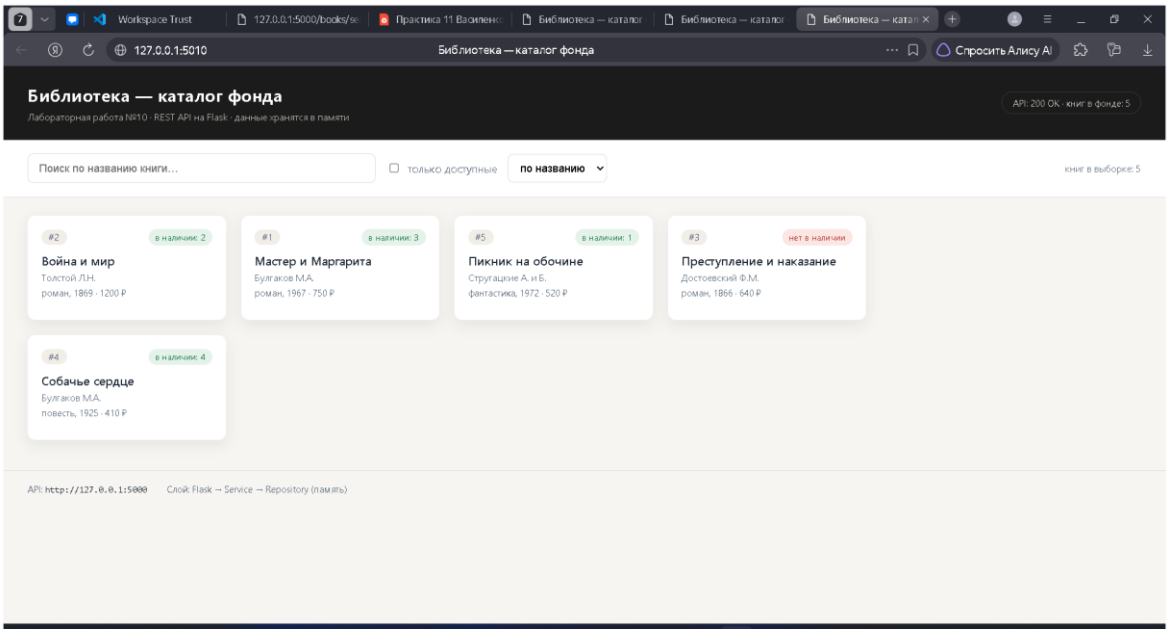


Рисунок 10.9 — Страница каталога, работающая поверх REST API (GET /)

Вывод

В ходе выполнения лабораторной работы изучено устройство протокола HTTP и архитектурный стиль REST, освоены методы GET, POST, PUT, PATCH и DELETE, коды состояния, заголовки, path- и query-параметры, а также формат обмена данными JSON.

Для предметной области «Библиотека» разработано клиент-серверное приложение: REST API на Flask с полным набором операций над ресурсом Book, фильтрацией, сортировкой, поиском по автору, валидацией входных данных и единым форматом ошибок, а также отдельный консольный HTTP-клиент на библиотеке requests. Приложение разделено на слои Route, Service и Repository, поэтому бизнес-логика не зависит от HTTP-уровня и может тестироваться отдельно.

Написано 19 автоматических тестов pytest, все тесты пройдены успешно. Разработанный API задокументирован таблицей endpoint, для каждой операции приведены примеры запросов и ответов. Полученные навыки являются основой для дальнейшего изучения баз данных, ORM и современных Python web-фреймворков.